

郝健:Linux下服务程序启动管理方式的分析与总结

原创 郝健 [Linux阅码场](#) 2020年01月20日 18:23

目前，Linux平台下主流的服务程序启动管理方式有以下几种：

1. daemon
2. sysvinit
3. systemd
4. nohup

1. daemon

守护进程是在后台运行不受控端控制的进程，通常情况下守护进程在系统启动时自动运行。

1.1 概念说明

进程组

每个进程除了有一进程ID(PID)之外，还属于一个进程组。进程组是一个或多个进程的集合。每个进程组有一个唯一的进程组ID(PGID)。进程组ID类似于进程ID，它是一个正整数，并可存放在pid_t数据类型中。函数getpgrp返回调用进程的进程组ID。

```
#include <sys/types.h>
#include <unistd.h>

pid_t getpgrp(void);

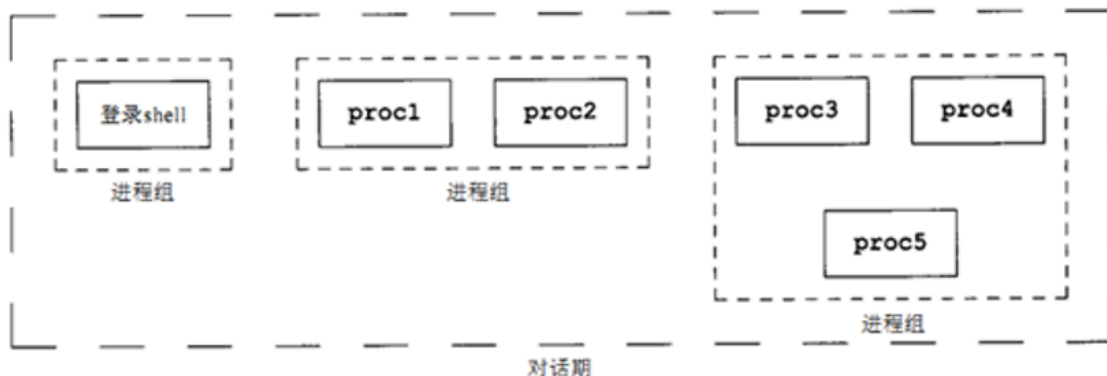
// Returns: 调用进程的进程组ID
```

会话期(session)

会话期是一个或多个进程组的集合。通常是由shell的管道将几个进程编成一组的。比如，下图中的安排 可能是由下列形式的shell命令形成的：

proc1 | proc2 &

proc3 | proc4 | proc5



一般，登陆一个shell时就会创建一个会话期。进程调用setsid函数就可建立一个新会话期。

```
#include <sys/types.h>
#include <unistd.h>

pid_t setsid(void);
// Returns: 若成功则为进程组ID，若出错则为-1。
```

如果调用此函数的进程不是一个进程组的组长，则此函数创建一个新会话期，结果为：

1. 此进程变成该新会话期的会话期首进程（session leader，会话期首进程是创建该会话期的进程）。此进程是该新会话期中的唯一进程。
2. 此进程成为一个新进程组的组长进程。新进程组ID是此调用进程的进程ID。
3. 此进程没有控制终端。如果在调用setsid之前此进程有一个控制终端，那么这种联系也被解除。如果此调用进程已经是一个进程组的组长，则此函数返回出错。为了保证不处于这种情况，通常先调用fork，然后使其父进程终止，而子进程则继续。因为子进程继承了父进程的进程组ID，而其进程ID则是新分配的，两者不可能相等，所以这就保证了子进程不是一个进程组的组长。

控制终端(terminal)

会话期和进程组有一些其他特性：

1. 一个会话期可以有一个单独的控制终端（controlling terminal）。这通常是我们在其上登录的终端设备（终端登录情况）或伪终端设备（网络登录情况）。
2. 建立与控制终端连接的会话期首进程，被称为控制进程（controlling process）。
3. 一个会话期中的几个进程组可被分成一个前台进程组（foreground process group）以及一个或几个后台进程组（background process group）。
4. 如果一个会话期有一个控制终端，则它有一个前台进程组，其他进程组则为后台进程组。
5. 无论何时键入中断键（常常是DELETE或Ctrl -C）或退出键（常常是Ctrl -\），就会造成将中断信号或退出信号送至前台进程组的所有进程。
6. 如果终端界面检测到网络已经断开连接，则将挂断信号送至控制进程（会话期首进程）。通常，我们不必关心控制终端，登录时，将自动建立控制终端。

1.2 创建方法

创建守护进程，应该创建一个进程然后将其放到一个新的会话期中。

1. 首先要做的是调用umask将文件模式创建屏蔽字设置为0。
2. 调用fork(子进程会是将来的守护进程)，然后使父进程退出(exit)，保证子进程不是进程组组长(这是setsid调用的必要前提条件)。
3. 调用setsid创建新的会话期。
4. 将当前目录改为根目录（如果不改为根目录可能导致不能umount某个目录）。
5. 关闭不再需要的文件描述符。

6. 某些守护进程打开/dev/null使其具有描述符0、1和2（将标准输入、标准输出、标准错误重定向 到/dev/null）。

```
int mydaemon()
{
    pid_t pid;

    umask(0); /* 1. 调用umask将文件模式创建屏蔽字设置为0. */
    pid = fork(); /* 2. 调用fork(), 然后使父进程退出(exit). */
    if (pid == -1)
        ERR_EXIT("fork error");
    if (pid > 0)
        exit(EXIT_SUCCESS);

    setsid(); /* 3. 创建新的session */

    if (chdir("/") == -1) /* 4. 将当前目录改为根目录 */
        ERR_EXIT("chdir");

    int i;
    for (i = 0; i < 3; i++)
        close(i); /* 5. 关闭不再需要的文件描述符, 这里关闭了0, 1, 2. */

    /* 6. 将标准输入, 标准输出, 标准错误重定向到/dev/null. */
    open("/dev/null", O_RDWR); /* 此时返回值为0, 则将标准输入重定向到了/dev/null. */
    dup(0); /* 使当前最小的空闲描述符1(标准输出)也指向0, 即/dev/null. */
    dup(0); /* 使当前最小的空闲描述符2(标准报错)也指向0, 即/dev/null. */
    return 0;
}
```

或者直接调用daemon函数

```
/*  
 * @nochdir: =0将当前目录更改至"/"  
 * @noclose: =0将标准输入、标准输出、标准错误重定向至"/dev/null"  
 * Returns: 若成功则为进程组ID, 若出错则为-1.  
 */  
int daemon(int nochdir, int noclose);
```

2. sysvinit

sysvinit主要依赖于Shell脚本，在/etc/rc.d/rc*.d建立软连接。在RedHat系列发行版中，还提供了 service, chkconfig 等命令行工具，来方便来管理 init 系统。sysvinit的主要问题是启动慢，已经逐渐 被淘汰。

目前Ubuntu和Centos等主流发行版都以移步到systemd，但仍然兼容老的写法，这里简单举个的例 子，编写一个simple脚本，添加执行权限后，保存到/etc/init.d/目录下。

```
#!/bin/sh
# A simple service

function start() {
    echo "Service starting..."
}

function stop() {
    echo "Service stop..."
}

# main
if [ $# != 1 ]; then
    echo "usage : $0 [start|stop|restart]"
    exit 1
fi

case $1 in
    start)
        stop > /dev/null 2>&1
        start
        ;;
    stop)
        stop
        ;;
    restart)
        stop
```



```
start
;;
esac
```

测试:

```
root ~ # service simple start
Service starting...
root ~ # service simple stop
Service stop...
root ~ # service simple restart
Service stop...
Service starting...
root ~ #
```

3. systemd

systemd是目前主流Linux发行版采用的init项目，systemd起初饱受争议，可以搜索“the software that currently breaks your audio”查看。但随着越来越稳定和使用C开发，高效启动，已经逐渐成为主角。Linus也表示：Torvalds says he has no strong opinions on systemd

<https://www.itwire.com/business-it-news/open-source/65402-torvalds-says-he-has-no-strong-opinions-on-systemd>

我们简单实现一个simple-server来感受一下systemd，主要步骤如下：

1. 编写一个需要作为守护的程序，如：simple-server.c

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int i = 0;
    while (1) {
        sleep(1);
        printf("hello world index: %d\n", i++);
    }

    return 0;
}
```

2. 编写service文件

```
[Unit]
Description=simple server

[Service]
ExecStart=/home/artist/code/simple-server/simple-server
Restart=always

[Install]
WantedBy=multi-user.target
```

具体的说明详见；

<https://www.freedesktop.org/software/systemd/man/systemd.service.html>

其中Restart=always代表在任何情况下service被杀死后，都需要自动重启，其他说明详见如上链接中的 Table 2。

3. `sudo cp simple-server.service /lib/systemd/system`
4. `sudo systemctl enable simple-server` 设置为开机启动，会自动创建软连接
`/etc/systemd/system/multi-user.target.wants/simpleserver.service` ->
`/lib/systemd/system/simple-server.service`
5. `sudo systemctl start simple-server` 启动

6. `sudo systemctl status simple-server` 查看状态
7. `cat /var/log/syslog` 查看日志输出
8. `ps tree | grep simple-server`
9. `sudo killall simple-server` 杀死服务，验证是否自动重启。

几点说明：

1. 修改`simple-server.service`后，需要执行`systemctl daemon-reload`重新加载。
2. 在自`daemon`的方式中，是无法决定服务程序信息输出的位置的，只能重定向到`/dev/null`，而`systemd`的方式可以灵活配置重定向的位置，`/var/log/syslog`为默认日志路径。
3. `service`文件中`Restart`选项体现了"子死父知因"的重要性。

4. nohup

一个terminal一旦关闭，其绑定的shell会收到`SIGHUP`信号，且shell在退出之前会将`SIGHUP`信号广播 给其上的进程组，即其上所有进程都会退出。这也是自`daemon`需要调用`setsid`脱离terminal的原因。`nohup`命令并不是真正脱离terminal，而是忽略`SIGHUP`。将标准输入重定向到`/dev/null`，标准输出和标准出错重定向到`nohup.out`，且不会随着terminal的关闭而退出。验证：

1) 正常启动一个死循环程序

```

artist ~ $ pidof a.out
2716
artist ~ $ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT     19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH   29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
artist ~ $ ps -C a.out s
  UID      PID          PENDING          BLOCKED          IGNORED          CAUGHT STAT TTY          TIME COMMAND
  1000    2716 0000000000000000 0000000000000000 0000000000000000 0000000000000000 S+   pts/0          1:26 ./a.out
artist ~ $ ls -l /proc/2716/fd
total 0
lrwx----- 1 artist artist 64 Jan 16 23:20 0 -> /dev/pts/0
lrwx----- 1 artist artist 64 Jan 16 23:20 1 -> /dev/pts/0
lrwx----- 1 artist artist 64 Jan 16 23:20 2 -> /dev/pts/0

```

2) nohup启动同样的死循环程序

```

artist ~ $ pidof a.out
2765
artist ~ $ ps -C a.out s
  UID      PID          PENDING          BLOCKED          IGNORED          CAUGHT STAT TTY          TIME COMMAND
  1000    2765 0000000000000000 0000000000000000 0000000000000000 0000000000000000 D+   pts/0          0:22 ./a.out
artist ~ $ ls -l /proc/2765/fd
total 0
l-wx----- 1 artist artist 64 Jan 16 23:23 0 -> /dev/null
l-wx----- 1 artist artist 64 Jan 16 23:23 1 -> /home/artist/nohup.out
l-wx----- 1 artist artist 64 Jan 16 23:23 2 -> /home/artist/nohup.out

```

对比总结

1. `daemon` 通过自实现`daemon`，可以清楚的看出一个守护进程实现的原理，方便理清概念。但缺点是在实际工程中默认将输出重定向到`/dev/null`，无法保存服务日志，排查服务问题，即使在程序中重定向到 某个日志目录，也只能写死，修改位置需要重新编译，不够灵活。
2. `sysvinit` `sysvinit`的优点是简单，只需要编写脚本。将服务添加到某个`runlevel`时，只需要创建软链接文件 即可，DevOps兴起之前，传统运维人员可以很快上手。顺序执行，方便排查问题。但顺序执行既是优点也是缺点，带来的弊端是运行效率慢，这也是逐渐被淘汰的主要原因。毕竟现在Linux不只 用于服务器系统，终端用户更追求`fast boot`的用户体验。
3. `systemd` `systemd`是目前主流Linux发行版中最新的初始化系统，主要的设计目标就是提高系统的启动速度。`.service`文件中包含很多选项，配置灵活，比如日志位置和重启方式等。当然，最主要的卖点 还是并行启动速度快。
4. `nohup` 在实际工程中，`nohup`也挺常用。比如一个类服务程序的调试，简单执行即可在一段时间通过 `nohup.out`文件观察问题，但其并不能算真正的守护程序。

综上，`systemd`看起来可以满足一个产品级别服务程序的两个最重要的需求：根据不同情况重启；记录 日志灵活可查，而这是`daemon`和`nohup`方式无法快速方便满足的。`sysvinit`简单清晰，但启动慢，就作为传统运维人员的美好记忆保留吧。
